

情報学群実験第3 最終レポート

学籍番号 1280391

細川 夏風

2026 年 5 月 20 日

1 背景と目的

近年、情報通信分野が発展している。中でも多くのデバイスがインターネットに接続されている。ネットワーク (以下: NW) は現在の情報通信分野の基盤を支えている。これから先、多くのエンジニアが NW の設計に携わるか、それに近い業務を行うことになると予想される。そして、設計には NW の理解が必要である。しかし、NW について深く理解することは非常に難解なことである。そのため、今回は Network Simulator2 (以下: NS2) というソフトを用いて実際の NW の動作、主に TCP/IP というネットワークアーキテクチャのトランスポート層プロトコルの動作とパケットの挙動について考察と議論を行う。

2 実験方法

今回、私は GID, SID はともに 1 である。また、パケット損失率、スループットは以下の定義から算出される [1]。

T := スループット, D := 送信データ量, t := 時間

$$T = \frac{D}{t}$$

W := ウィンドウサイズ, RTT := 往復遅延時間

$$T = \frac{W}{RTT}$$

L := パケット損失率, N_{drop} := ドロップしたパケット数, N_{sent} := 送信したパケット数

$$L = \frac{N_{drop}}{N_{sent}} \times 100$$

また、今回用いるソフトウェアとそのバージョンは以下のとおりである。

Gnuplot

```
1 $ gnuplot --version
2 gnuplot 6.0 patchlevel 0
```

Network Simulator2

```
1 $ ns
2 % puts [ns-version]
3 2.35
```

問題 1

ボトルネックリンクの帯域幅 BW を 1 Mbps から 10 Mbps まで変化させ、スループットおよびパケット損失率を測定した。

また、パケット損失率が 1.1% を初めて超える帯域幅を Efficiency Threshold とした。

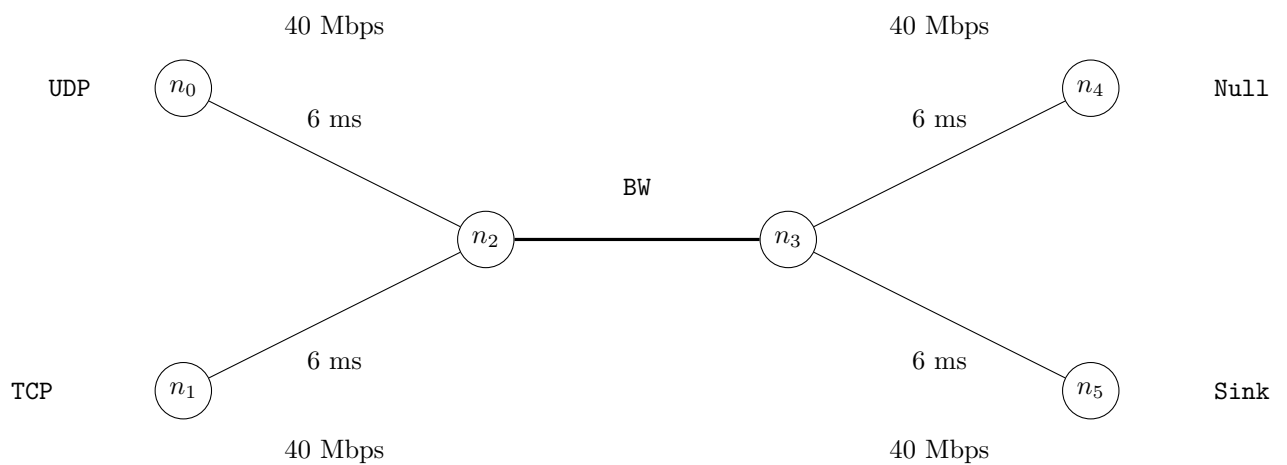


図 1 問題 1 のネットワーク構成

問題 2

ルータのバッファサイズ B を 1 kbytes から 10 kbytes まで変化させ、パケット損失率およびキューイング遅延を測定した。

さらに、バッファサイズ増加時におけるパケット損失率の変化について解析を行った。

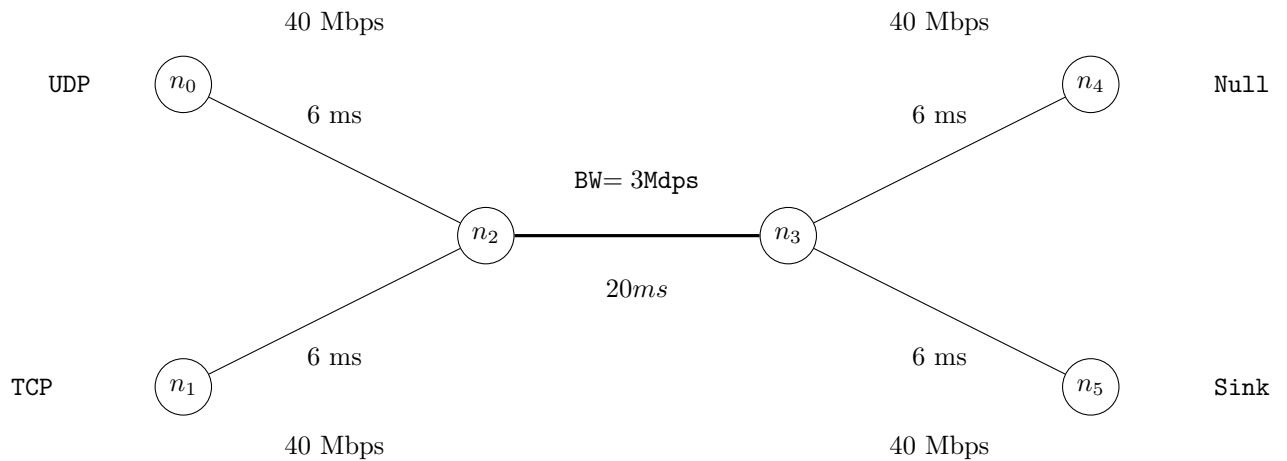


図 2 問題 2 のネットワーク構成

問題 3

TCP Tahoe を用いて通信を行い、 $cwnd$ および $ssthresh$ の時間変化を観測した。

また、最大ウィンドウサイズおよび送信スループットを算出した。

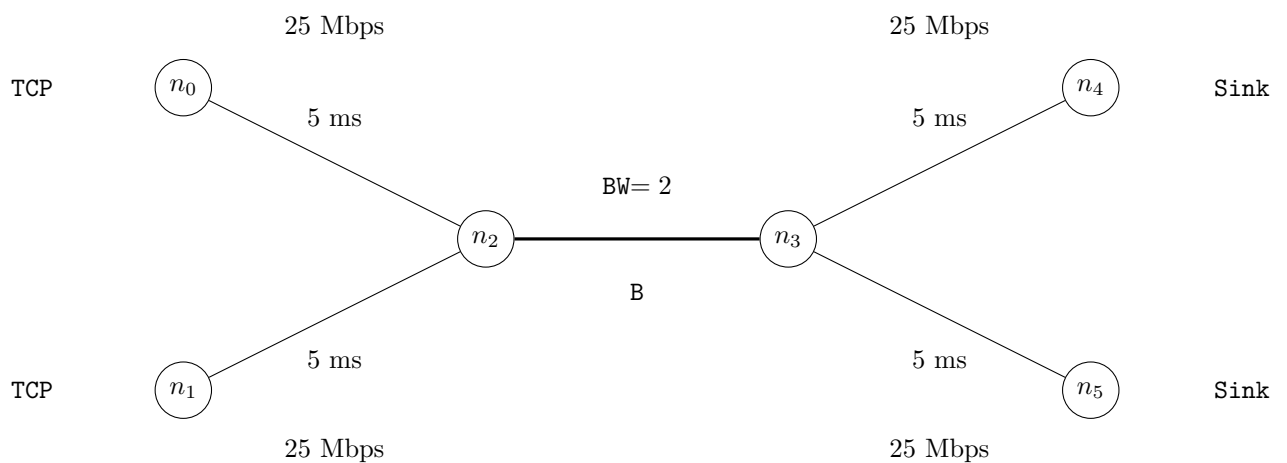


図 3 問題 3 のネットワーク構成

問題 4

TCP Reno を用いて通信を行い, $cwnd$ および $ssthresh$ の変化を TCP Tahoe と比較した.
また, 輻輳によるパケット損失数およびシーケンス番号を確認した.

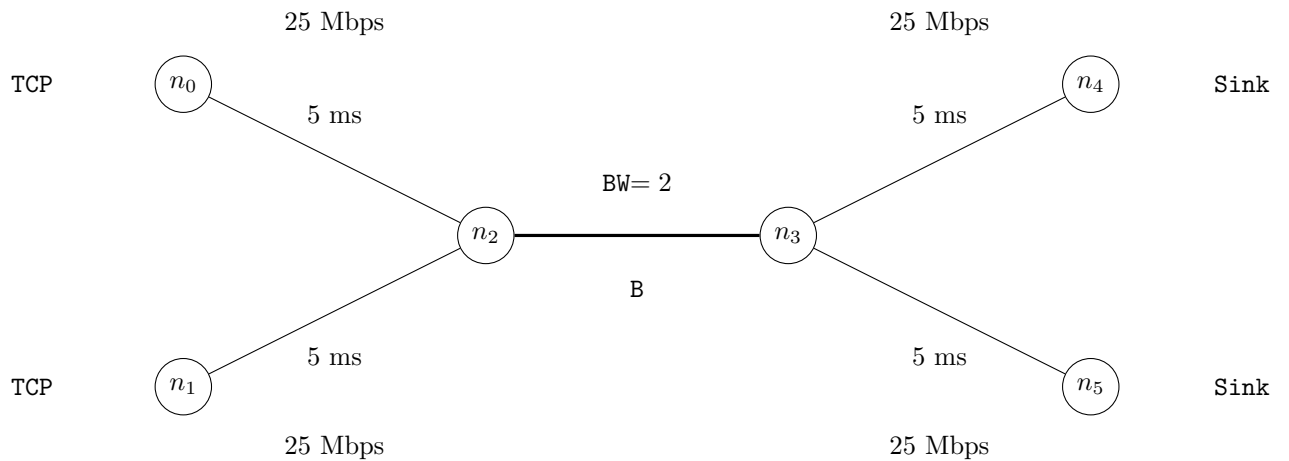


図 4 問題 4 のネットワーク構成

3 結果

問題 1-1

図 5 にスループットと帯域幅の関係を示す。

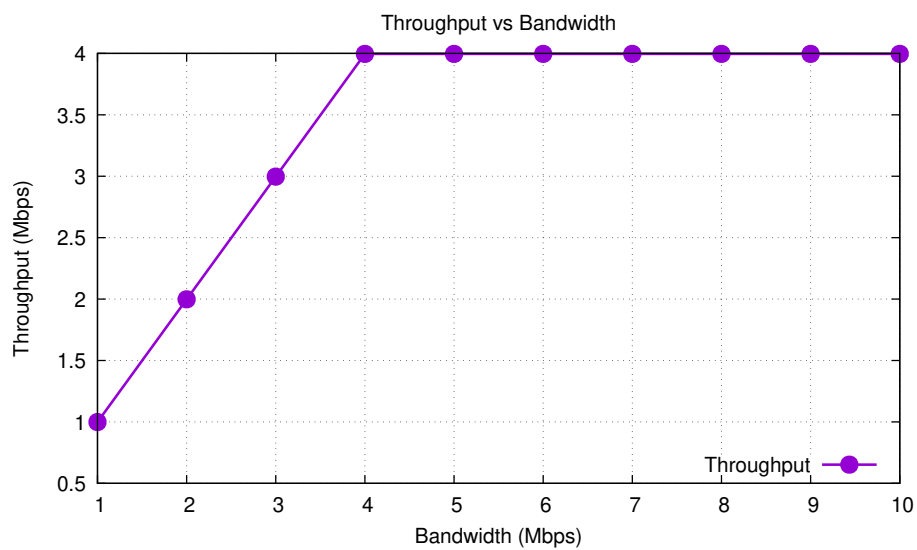


図 5 スループットと帯域幅の関係

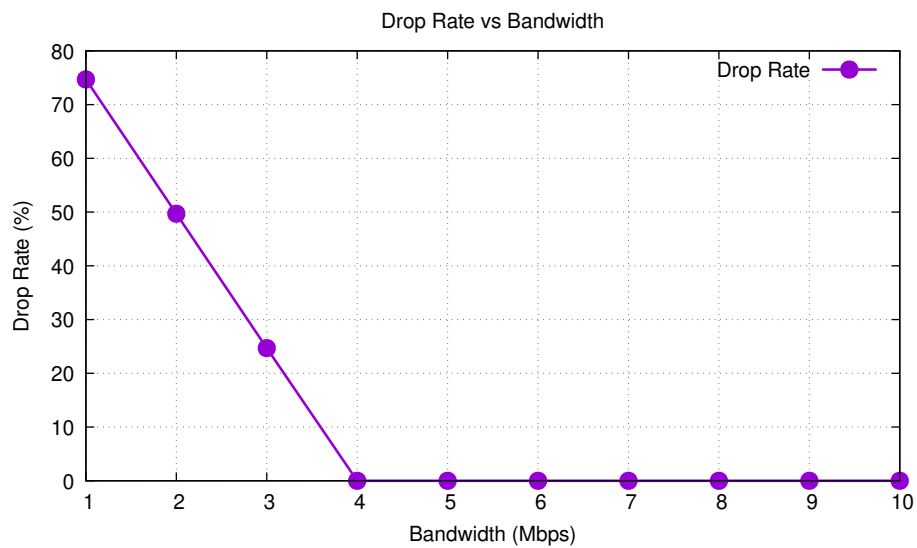


図 6 パケット損失率と帯域幅の関係

問題 1-2

付録 1 から得られる値は次ようになる。

表 1 帯域幅に対するスループットおよびパケット損失率の測定結果

帯域幅 (Mbps)	スループット (Mbps)	パケット損失率 (%)
1	0.998800	74.682405
2	1.997999	49.694908
3	2.996999	24.687406
4	3.995998	0.000000
5	3.996079	0.000000
6	3.996132	0.000000
7	3.996170	0.000000
8	3.996199	0.000000
9	3.996221	0.000000
10	3.996239	0.000000

以上の結果から、BW が 3Mbps のときに Efficiency Threshold を下回っている事がわかる。よって、Efficiency Threshold は 3Mbps である。

問題 1-3

上記のデータ群から一定値を超えるとスループットが一定値で飽和していることがわかる。これは、送信データ量より帯域幅が大きくなったためである。1 秒間に送信可能なデータ量が実際に送

信しているデータ量よりも大きい場合、パケットが送信中にバッファから溢れてドロップすることがなくなるのだ。よって、パケット損失率も 0% であり、スループットが一定値で飽和している事がわかる。

問題 2-1

図 7 にバッファサイズとパケット損失率の関係を示す。

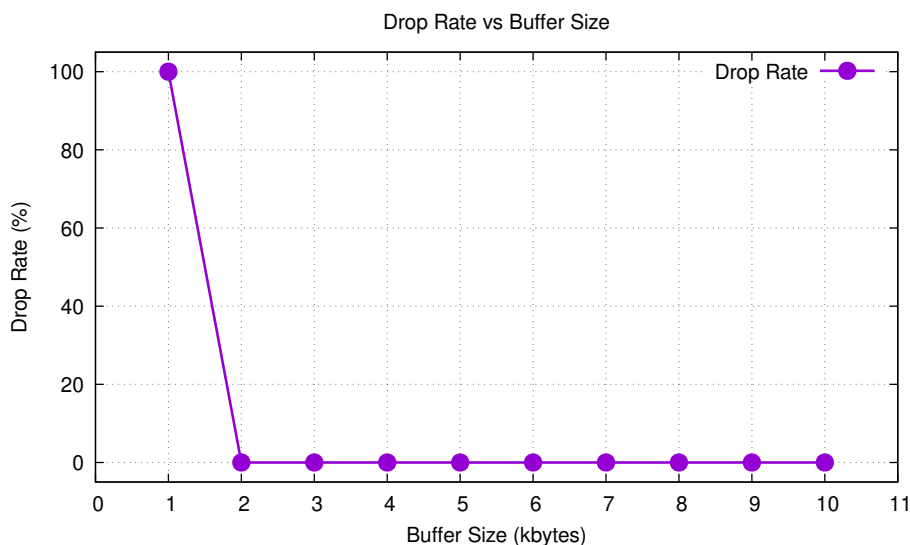


図 7 バッファサイズとパケット損失率

問題 2-2

付録 3 から、その結果は 0 であった。よって、最大バッファサイズにおけるキューイング遅延は無いと言える。また、これはデータからも考えられる。表 1 でも最大バッファサイズのパケット損失率は 0% であり、送信データ量に対して帯域幅が大きいから起こっているからであると考えられる。そのため、バッファに格納される時間が著しく短いか、ほとんど無いと言える。

問題 2-3

バッファサイズとパケット損失率の関係を以下のグラフに示す。このデータは意図的に帯域幅を小さく設定し、パケット損失率が大きくなるように設定している。

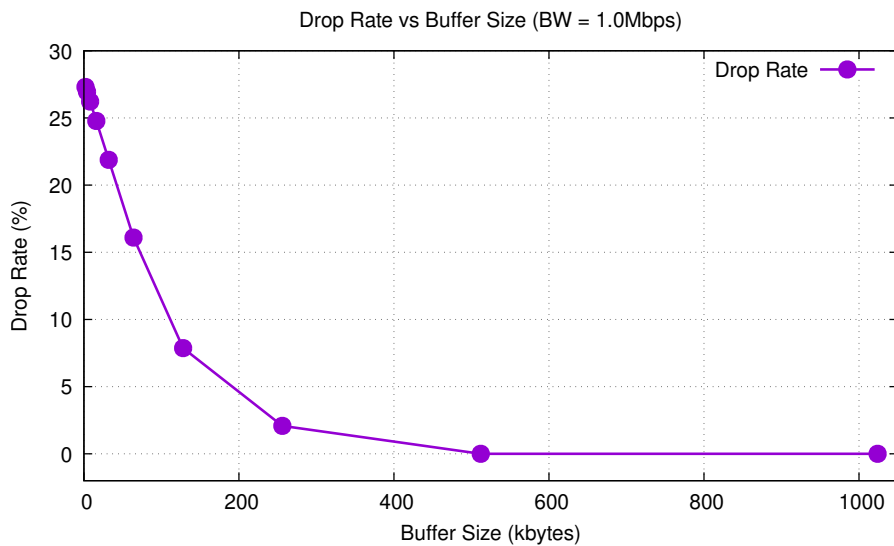


図 8 バッファサイズとパケット損失率

問題 3-1

cwnd(送信ウィンドウサイズ), ssthresh(スロースタート閾値) の時間変化を以下のグラフに示す。

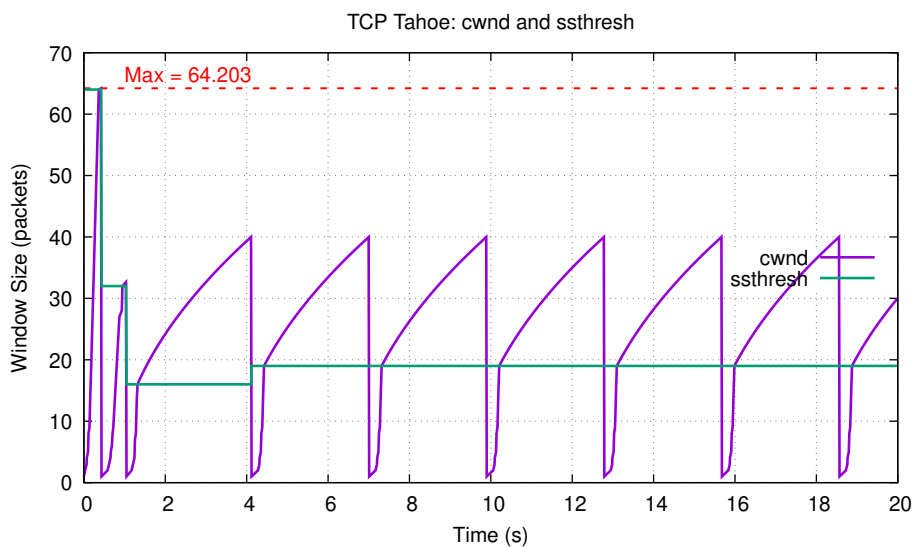


図 9 cwnd と ssthresh の時間変化におけるグラフ

問題 3-2

付録 5 の結果から値は 0.029091 であった。よって、11ms のまでスループットは 0.029091Mbps である。

問題 3-3

図 9 から、ssthresh は一度減少したあとわずかに上昇したように見える。

問題 4-1

cwnd(送信ウィンドウサイズ), ssthresh(スロースタート閾値) の時間変化を以下のグラフに示す。

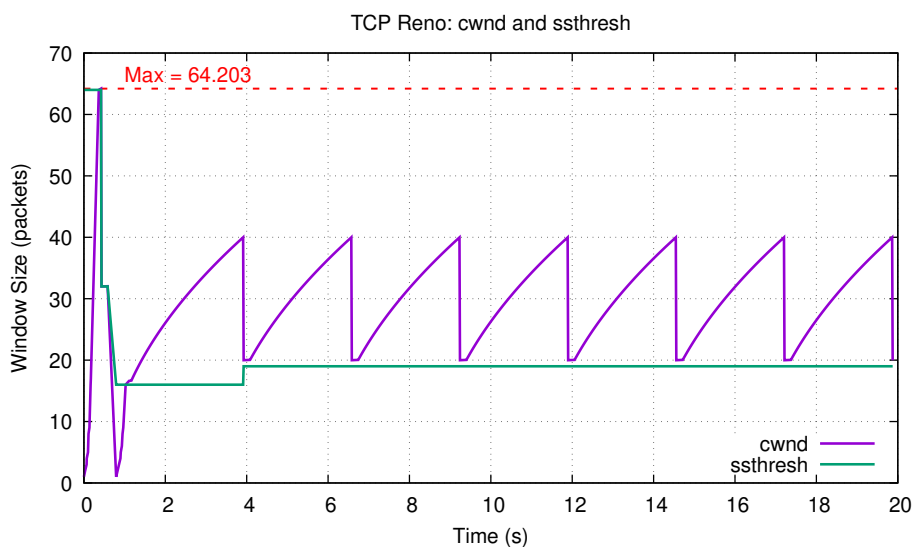


図 10 cwnd と ssthresh の時間変化におけるグラフ

問題 4-2

付録 6 の結果から、出力された値は 0.029091 である。よって 11ms までのスループットは 0.029091Mbps である。

問題 4-3

付録 7 の結果から、出力された値の集合は

$L = \{76, 78, 80, 82, 84, 86, 88, 90, 92, 94, 96, 98, 100, 102, 104, 106, 108, 110, 112, 114, 116, 118, 120, 122, 124, 126, 801, 1440, 2079, 2718, 3357, 3996, 4635\}$

である。これはプログラムからドロップしたシーケンス番号の集合である。この集合の濃度 $|L|$ は 33 である。

4 考察

問題 1

この問ではボトルネックリンクにおける帯域幅がパケットの損失率とスループットに与える影響を示している。図 5, 6, 表 1 から帯域幅の上昇はパケットの損失率を減少させ、スループットを上昇させる効果があることがわかる。その上昇量はパケット損失率は割合のためもちろん 0% で停止する。それに加えて、スループットもある帯域幅から極端に上昇量が小さくなるか 0 になる場合がある。これは 3 節にもあるが、送信しているデータ量に比べて帯域幅が大きくなったためであると考えられる。

問題 2

この問ではバッファサイズがパケット損失率、遅延に与える影響を示している。図 7, 8 より、バッファサイズを上昇させることによってパケット損失率が小さくなるか、0% になっていることがわかる。これはホップのバッファ内に格納できるパケットの量が多くなっているため、溢れてドロップするという事が少なくなるためであると考えられる。最大バッファサイズにおける遅延はほとんど無いという結果になった。しかし、これは帯域幅がもっと小さくかつバッファサイズが十分に大きい場合、バッファ内に格納される時間が大きいため遅延時間が長くなるというケースが考えられる。

問題 3

この問では Tahoe における $cwnd$ と $ssthresh$ の時間における推移を示している。図 9 から $ssthresh$ は一度減少した後、わずかに上昇している。これは最適化の結果である。Tahoe では、輻輳を検知すると $ssthresh$ の $\frac{1}{2}$ 倍し、 $cwnd$ を 1 にして通信を再開する。この $ssthresh$ が上昇すると指数的に上昇する時間がわずかに増加するため、その分パケットが多く送られることになる。よって、単位時間あたりの送信されるパケットが増加すると考えられる。

問題 4

この問では Reno における $cwnd$ と $ssthresh$ の時間における推移を表している。図 10 から $ssthresh$ は一度に減少したあと、上昇している。この方式でも 4 節と同じことが言える。つまり、最適化の結果であると考えられる。しかし、Reno は輻輳を検知すると $cwnd$ を $\frac{1}{2}$ 倍して通信を再開する。これはあくまでパケットの受信順番の入れ違いで発生してしまった輻輳だと検知し、大きく $cwnd$ を落とさずに高速な通信を維持する方法である [2]。しかし、3 節から、合計 33 回はドロップが発生していることがわかる。高速に通信を行おうとすれば必然的にドロップ回数は多くなってしまいうという事がわかる。これは通信の帯域幅の限界に近い部分まで通信を行うというアルゴリズムが作動しているからとも言える。

参考文献

- [1] パケットキャプチャ実践技術 Wireshark によるパケット解析 実践編, 竹下恵, 株式会社リックテレコム, 2012 年 12 月 5 日
- [2] M. Allman, V. Paxson, and E. Blanton, "TCP Congestion Control," RFC 5681, Sep. 2009. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5681>

5 作成したシェルスクリプト

問題 1 の自動実行とデータ抽出のために作成したシェルスクリプト (kadai1-1.sh) を以下に示す。

Listing 1 問題 1 の実行スクリプト (kadai1-1.sh)

```
1  #!/bin/sh
2  for bw in 1 2 3 4 5 6 7 8 9 10; do
3      ns kadai1-g1.tcl ${bw}Mb > /dev/null 2>&1
4      awk -v bw=${bw} '
5          BEGIN { st = -1 }
6          $3 == 2 && $4 == 3 {
7              if (st == -1) st = $2; et = $2
8              if ($1 == "+") sent++
9              if ($1 == "d") drop++
10             if ($1 == "r") recv_pkts++
11         }
12         END {
13             printf "%d %.6f %.6f\n", bw, (recv_pkts * 1000 * 8) / (et - st) /
14                 1000000, (sent > 0 ? (drop / sent) * 100 : 0)
15         }
16     ' out.tr
done
```

Listing 2 問題 2-1 の実行スクリプト (kadai2-1.sh)

```
1  #!/bin/sh
2  # 帯域幅は3Mbpsで固定、バッファサイズ(b)を1から10まで変化させる
3  for b in 1 2 3 4 5 6 7 8 9 10; do
4      ns kadai1-2-g1.tcl 3Mb ${b} > /dev/null 2>&1
```

```

5  awk -v b=${b} '
6      BEGIN { st = -1 }
7      $3 == 2 && $4 == 3 {
8          if (st == -1) st = $2; et = $2
9          if ($1 == "+") sent++
10         if ($1 == "d") drop++
11         if ($1 == "r") recv += $6
12     }
13     END {
14         printf "%d %.6f %.6f\n", b, (recv * 8) / (et - st) / 1000000, (sent
15             > 0 ? (drop / sent) * 100 : 0)
16     }
17     ' out.tr
done

```

Listing 3 問題 2-2 の実行スクリプト (kadai2-2.sh)

```

1  #!/bin/sh
2  ns kadai1-2-g1.tcl 3Mb 10 > /dev/null 2>&1
3
4  awk '
5      BEGIN { m = 0 }
6      $1 == "+" && $3 == 2 && $4 == 3 { eq[$12] = $2 }
7      $1 == "-" && $3 == 2 && $4 == 3 {
8          if ($12 in eq) {
9              d = $2 - eq[$12]
10             if (d > m) m = d
11         }
12     }
13     END {
14         printf "%.6f\n", m + 0
15     }
16     ' out.tr

```

Listing 4 問題 2-3 の実行スクリプト (kadai2-3.sh)

```

1  #!/bin/sh
2
3  for b in 2 4 8 16 32 64 128 256 512 1024; do

```

```

4 ns kadai1-2-g1.tcl 1.0Mb ${b} > /dev/null 2>&1
5
6 awk -v b=${b} '
7     $3 == 2 && $4 == 3 {
8         if ($1 == "+") sent++
9         if ($1 == "d") drop++
10    }
11    END {
12        printf "%d\t\t\t%.6f\n", b, (sent > 0 ? (drop / sent) * 100 : 0)
13    }
14    ' out.tr
15 done

```

Listing 5 問題 3-2 の実行スクリプト (kadai3-2.sh)

```

1 #!/bin/sh
2 TIME=0.011
3
4 awk -v t=$TIME '
5     BEGIN { s = 0 }
6     $1 == "+" && $3 == 2 && $4 == 3 && $5 == "tcp" && $2 <= t {
7         s += $6 * 8
8     }
9     END {
10        if (s > 0) {
11            printf "%.6f\n", (s / t) / 1000000
12        } else {
13            print "0.000000"
14        }
15    }
16    ' out.tr

```

Listing 6 問題 4-2 の実行スクリプト (kadai4-2.sh)

```

1 #!/bin/sh
2 TIME=0.011
3
4 awk -v t=$TIME '
5     BEGIN { s = 0 }

```

```

6  $1 == "+" && $3 == 2 && $4 == 3 && $5 == "tcp" && $2 <= t {
7      s += $6 * 8
8  }
9  END {
10     if (s > 0) {
11         printf "%.6f\n", (s / t) / 1000000
12     } else {
13         print "課題2: 0.000000 Mbps"
14     }
15 }
16 ' out.tr

```

Listing 7 問題 4-3 の実行スクリプト (kadai4-3.sh)

```

1  #!/bin/sh
2  awk '
3      BEGIN { count = 0; seq_list = "" }
4      $1 == "d" && $5 == "tcp" {
5          count++
6          seq_list = seq_list $11 " "
7      }
8      END {
9          if (count > 0) {
10             print "ドロップ総数: " count
11             print "シーケンス番号: " seq_list
12          } else {
13             print "ドロップしたパケットはありません。"
14          }
15      }
16  ' out.tr

```